PRISM

Technical Design Document

Cold-chain logistics operations assistant

Purpose	Current implementation guide
System	PRISM Streamlit application
Audience	Project reviewers, developers, and operations stakeholders
Scope	Implemented repository behavior as inspected on 11 October 2026

This document describes the code that exists in the project. It distinguishes implemented behavior from planned architecture and external services.

1. Executive overview

PRISM is a Python and Streamlit prototype that lets a dispatcher explore a local fleet dataset, test three integrations, and ask a conversational assistant to investigate logistics questions. The assistant can call a MySQL telemetry tool, an Open-Meteo weather tool, and a Pinecone-backed SOP search tool. A configured Groq or Gemini model decides when to call tools and writes the final response.

The application supports investigation and decision support. The code does not dispatch vehicles, change database records through the telemetry tool, or establish a live IoT ingestion pipeline.

1.1 Goals

Provide a simple UI for historical fleet summaries; retrieve selected telemetry through a restricted SQL view; retrieve relevant SOP text using vector search; fetch weather at supplied coordinates; and combine tool results into readable operational guidance.

1.2 Current boundaries

Area	Current behavior
Fleet dataset	The explorer expects a local CSV at Data/raw/dynamic_supply_chain_logistics_dataset.csv and labels it historical data.
SQL	The agent queries FDE_VIEWS.VW_ACTIVE_FLEET through a MySQL account. Tool output is limited to ten returned rows.
SOP RAG	Local Hugging Face embeddings create query vectors; Pinecone stores/searches the SOP chunks.
Weather	Open-Meteo current weather is queried for provided latitude/longitude. Disruption score is a simple wind threshold.
Audit	SQL defines an audit table and grants, but the application does not currently write audit events.
Deployment	This repository contains no Dockerfile or Compose definition. Database hosting may be external to this repository.

2. Architecture

PRISM runs as one Streamlit application. The UI calls local Python functions, while those functions connect to external services. The language model receives tool definitions through LangChain and LangGraph; it does not hold direct database credentials as a database client.

Layer	Main file(s)	Responsibility
User interface	app.py	Navigation, CSV explorer, explicit tool checks, chat history display, safe response display.
Agent orchestration	src/orchestration.py	Selects Groq or Gemini, binds tools, runs a LangGraph reasoner/tool loop, keeps in-memory thread checkpoints.
Tool implementations	src/agent_tool.py	Executes SELECT queries, requests weather, retrieves SOP passages.
Agent guidance	src/prompts/system_prompts.txt	Requests tool use and prescribes business response structure.
Relational setup	source/security_and_view.sql; source/audit_log.sql	Creates the fleet view and audit table/grants.
Fleet loader	source/inject_data.py	Maps CSV columns to legacy table names and replaces the raw table contents.
SOP loader	source/inject_spo.py	Parses supported policy files, chunks text, embeds, and upserts vectors to Pinecone.

2.1 Request flow

Step	What happens
1	A user enters a question in the Dispatch Assistant.
2	app.py loads the agent, creates/uses a thread ID, and adds the system prompt on the first turn.
3	LangGraph calls the configured model with the conversation state and tool definitions.
4	If the model requests a tool, ToolNode runs that Python tool. The result returns to the model.
5	The graph repeats model/tool steps until the model returns a final message.
6	Streamlit renders the response after a secret-redaction pass.

3. Fleet data and SQL

3.1 CSV to raw table

source/inject_data.py reads the configured logistics CSV, selects nine source columns, renames them to legacy field names, adds SYS_INGEST_FLAG, then writes TBL_SC_FLEET_HIST_RAW in the logistics schema. SQLAlchemy/PyMySQL connects using environment variables from the project .env. The `if_exists="replace"` option replaces the target table when the loader runs, so this script is a batch refresh rather than incremental live ingestion.

CSV field	Raw SQL field
timestamp	TS_UTC
vehicle_gps_latitude / vehicle_gps_longitude	V_LAT / V_LON
iot_temperature	IOT_TEMP_VAL_C
cargo_condition_status	CGO_COND_CD
risk_classification	RISK_CLS_TXT
delay_probability	DELAY_PROB_DEC
port_congestion_level	PRT_CNG_LVL
route_risk_level	RT_RSK_IDX

3.2 Raw schema and view schema

logistics contains the raw fleet table. FDE_VIEWS contains VW_ACTIVE_FLEET, a view that selects and renames chosen columns from the raw table. The view currently selects all source rows; it does not contain a WHERE clause that limits results to active records despite its name.

3.3 Query and response behavior

The SQL tool receives a SQL string from the model, rejects it unless its trimmed text begins with SELECT, connects to the configured FDE_VIEWS database, executes it, fetches at most ten rows, and returns column names plus row values as text. Errors become a “Database Error” string for the model/UI. The SELECT prefix check is a secondary guard; MySQL grants and view permissions are the security boundary.

The view exposes Timestamp, Latitude, Longitude, Current_Temperature_C, Cargo_Condition_Code, Risk_Classification, Delay_Probability, Port_Congestion_Level, and Route_Risk_Index.

4. SOP retrieval and RAG

4.1 Index preparation

source/inject_spo.py accepts Markdown, TXT, PDF, CSV, and XLSX policy files. Markdown headers are retained as metadata context; text is split into chunks of about 600 characters with 60 characters of overlap. Table files are represented one row at a time. A local Hugging Face model, by default BAAI/bge-small-en-v1.5, produces normalized 384-dimensional vectors. The script writes the text, metadata, and vectors to the Pinecone index fde-sop-index-local using cosine similarity.

The ingestion script tracks file hashes in a local JSON cache to skip unchanged files and delete/rebuild vectors for changed or removed documents. The SOP query tool lazily creates the embedding model and retriever on first search and asks Pinecone for the top two matching chunks.

4.2 Query-time retrieval

Stage	Data movement
Question	Dispatcher asks about an operational procedure.
Embedding	The same configured local embedding model converts the query into a vector.
Vector search	Pinecone ranks stored chunk vectors by similarity.
Context	The tool returns up to two matching text chunks and source metadata.
Answer	The LLM uses retrieved clauses when composing its response.

4.3 SOP example

The project SOP says fresh perishables should remain from 0.0°C to 4.0°C. Above 4.0°C is a critical breach. It directs the dispatcher to contact the driver and restart the auxiliary cooling unit; if ETA delay is greater than one hour, divert to the nearest emergency cold-storage facility. High Risk together with delay probability greater than 0.65 is escalated to the Tier 2 Logistics Manager.

5. Weather and agent behavior

5.1 Corridor conditions

fetch_corridor_conditions calls the Open-Meteo forecast endpoint using the provided coordinates and requests current weather. It returns external temperature and wind speed. The code assigns a congestion index of 8.5 when wind speed is above 10 km/h, otherwise 2.5. This index is a project heuristic; it is not a measured road-traffic or port-congestion feed.

5.2 Orchestration

src/orchestration.py uses Agent_llm to select Groq (default) or Gemini. It binds the SQL, weather, and SOP functions as callable tools. The LangGraph state contains message history, and MemorySaver stores checkpoints in process memory under a thread ID. In app.py the system prompt is injected the first time a thread is used. Subsequent requests use that graph thread state.

The system prompt says to use telemetry, weather, and SOP search, then format an executive summary, analysis table, action plan, and SOP citation. Tool selection still occurs through model decisions; the prompt is guidance, not an enforcement mechanism that guarantees every tool runs for every question.

5.3 User-facing screens

Screen	Behavior
Overview	Explains available project capabilities without initializing backend integrations.
Data explorer	Reads the expected local CSV on demand, caches it, and displays counts, charts, and a sampled location map. It labels records as historical, not live.
Tool checks	Runs MySQL, weather, or SOP search independently so a service can be checked in isolation.
Dispatch assistant	Sends a question to LangGraph and renders its final response in the chat.

6. Security and governance

6.1 Relational least privilege

security_and_view.sql creates prism_view_owner with SELECT on the raw table, then defines VW_ACTIVE_FLEET with SQL SECURITY DEFINER and grants usr_fde_ro SELECT on the view. source/audit_log.sql creates FDE_VIEWS.AgentAuditLog and grants usr_fde_ro INSERT and SELECT on that table. These grants separate fleet reading from audit-table insertion.

The SQL tool checks for a SELECT prefix, but SQL grants are the enforceable permission boundary. The current view is a projection of chosen columns, not a row-level active-shipment filter. The SQL scripts use a broad host wildcard (%) and include a literal password in the view-owner setup; those are prototype configuration choices to harden before production.

6.2 Audit implementation status

The audit SQL defines fields for timestamp, session, node, tool, and content. It does not make auditing happen by itself. The current app/tool code has no INSERT call to AgentAuditLog, so do not describe each agent decision as automatically recorded in the audit table.

6.3 Secrets and service boundaries

Application secrets are loaded from the project .env and should remain outside source control. app.py scrubs detected keys and database connection strings from displayed error text. MySQL is accessed at the configured host and port, which default in code to localhost and 3307. A Docker-run MySQL container can provide that local service, but this codebase does not define Docker orchestration.

The language model provider, Pinecone, Open-Meteo, and MySQL are separate availability and data-sharing boundaries. Operational users should verify high-impact recommendations against source telemetry and the approved SOP.

7. Operations and configuration

7.1 Configuration categories

Environment setting	Purpose
MYSQL_HOST / MYSQL_PORT	MySQL endpoint; source defaults are localhost and 3307.
MYSQL_USER / MYSQL_PASSWORD / MYSQL_DATABASE	Credentials and schema. Loader defaults differ from the read-only tool account.
PINECONE_API_KEY	Pinecone API authentication.
GROQ_API_KEY or GEMINI_API_KEY	Credentials for the selected model provider.
Agent_llm	Provider selector: GROQ by default or GEMINI.
EMBEDDING_MODEL_NAME	Local embedding model name; defaults to BAAI/bge-small-en-v1.5.

7.2 Setup sequence at a high level

Order	Action
1	Start or provision MySQL and create required schemas/tables.
2	Run security_and_view.sql and audit_log.sql with an administrative database account.
3	Place the expected fleet CSV and policy files at their configured project paths.
4	Run source/inject_data.py to load the CSV into the raw fleet table.
5	Run source/inject_spo.py to create/update the Pinecone SOP index.
6	Configure .env values for database, model provider, and Pinecone.
7	Start the Streamlit app; use Tool Checks to validate each integration separately.

This sequence describes the code's external dependencies. The repository snapshot does not include a Dockerfile or Compose file, so container startup is not managed by the project files shown here.

8. Limitations and production considerations

Topic	Observed implementation	Practical implication
Data freshness	The loader replaces the raw table from a CSV; the explorer reads local CSV separately.	Results can be historical or differ between SQL and the explorer until refreshed consistently.
View semantics	VW_ACTIVE_FLEET selects rows from the raw table without an active-status filter.	Do not infer “currently active” from the view name alone.
Query controls	Tool checks only whether SQL begins with SELECT.	Use database grants and query limits as primary controls; validate query behavior.
Audit	Audit table/grants exist, but code does not append entries.	No end-to-end agent audit trail is currently produced by this app.
Weather heuristic	High disruption is inferred from wind speed above 10 km/h.	It is an illustrative rule, not road traffic or port status.
RAG evidence	The retriever returns the top two semantically similar chunks.	Review source text and coverage; retrieval can omit the relevant clause.
Deployment	No app Dockerfile/Compose definition is in the inspected repository.	Container deployment needs separate packaging and operations configuration.
Human oversight	The assistant presents recommendations and the user sees the response.	Confirm shipment state and approved procedures before operational action.

8.1 Suggested hardening sequence

First verify and version the fleet source and confirm the view’s intended row filter. Then replace literal credentials with managed secrets, restrict database network access, and test least-privilege grants. Wire structured audit writes with retention and access controls. Finally validate the weather heuristic, add service-health and retrieval checks, and define reproducible application/database deployment.

9. Source map

Project file	Documented responsibility
app.py	Streamlit pages, CSV explorer, tool checks, chat flow, secret-safe display.
src/agent_tool.py	SQL, weather, and SOP retrieval tools.
src/orchestration.py	LangGraph and model/tool loop.
src/prompts/system_prompts.txt	Agent guidance and response format.
source/inject_data.py	CSV-to-MySQL batch load.
source/inject_spo.py	Policy parsing, chunking, embeddings, and Pinecone sync.
source/security_and_view.sql	Restricted fleet view and account grants.
source/audit_log.sql	Audit table definition and grants.
Data/policy/cold_chain_sop.md	Operational limits, breach response, diversion, and escalation rules.